# TCP Congestion Avoidance: TCP Tahoe vs. TCP Reno

Practice Questions and Textbook Problems 32–44 (Chapter 3)

## Background (quick reference)

- **AIMD:** additive increase (about +1 MSS per RTT) and multiplicative decrease (window halved on loss).
- **State variables:** congestion window `cwnd`, slow-start threshold `ssthresh`.
- **Slow start:** `cwnd` doubles every RTT (one MSS per ACK) until it reaches `ssthresh`.
- **Congestion avoidance:** once `cwnd` $\geq$ `ssthresh`, `cwnd` grows by about 1 MSS per RTT.
- **Two loss signals:**
  - *Timeout* (severe congestion): both Tahoe and Reno set `ssthresh` = `cwnd`/2, `cwnd` = 1 MSS, and re-enter slow start.
  - *Triple duplicate ACK* (mild congestion): **Tahoe** treats it like a timeout (`cwnd` = 1, slow start). **Reno** performs fast recovery: `ssthresh` = `cwnd`/2, `cwnd` = `ssthresh` +3 MSS, inflates by 1 MSS per further duplicate ACK, then deflates to `ssthresh` and continues in congestion avoidance.

## Part A: Conceptual Questions

A1. Why does TCP use AIMD rather than a fixed window? What would happen to fairness if all flows used a fixed window larger than the bottleneck capacity?

A2. Explain why TCP cannot know the network's available bandwidth and therefore must *probe* for it. What signal does it use to infer congestion?

A3. Why is `ssthresh` set to half of the *current* window rather than to a fixed value? Must the actual sending rate at the time of loss equal `cwnd` segments per RTT? Justify.

A4. Why does a triple duplicate ACK indicate a *milder* form of congestion than a timeout? What does a duplicate ACK tell the sender that a timeout does not?

A5. Explain why Reno inflates `cwnd` by one MSS for each additional duplicate ACK during fast recovery. What would the sender do without this inflation?

A6. Explain why, in congestion avoidance, `cwnd` grows by roughly one MSS per RTT even though the sender receives one ACK per segment. Where does the "per RTT" behaviour come from?

A7. How does a *delayed ACK* (one ACK for every two segments) change the growth rate of `cwnd` in slow start and in congestion avoidance?

A8. Why does a timeout require the sender to fall back to slow start instead of continuing in congestion avoidance? What information is lost after a timeout?

A9. Why might Tahoe and Reno perform similarly when losses are rare but differ sharply when losses are frequent and isolated?

A10. Explain why AIMD flows sharing a bottleneck converge toward an equal share of bandwidth, and why this relies on the *multiplicative* decrease step (see Textbook Problem 41).

## Part B: Tahoe vs. Reno in Different Circumstances

Assumptions for Figure 1: one RTT per transmission round; initial `ssthresh` = 16; fast recovery is simplified so that `cwnd` = `ssthresh` after recovery; both protocols see the same losses: a triple duplicate ACK in round 8 and a timeout in round 16.

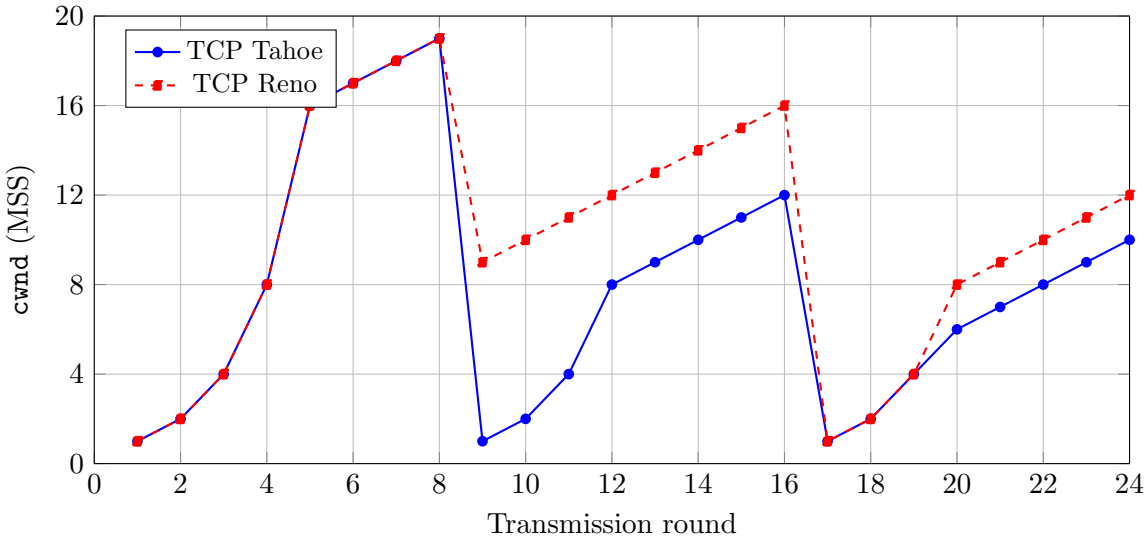



Figure 1: Sample `cwnd` evolution for Tahoe and Reno. Triple duplicate ACK in round 8; timeout in round 16.

B1. Using Figure 1, identify the rounds in which each protocol is in slow start and in congestion avoidance. Do the two protocols differ in the rounds 1–8?

B2. Compute `ssthresh` and `cwnd` for Tahoe and for Reno immediately after the triple duplicate ACK in round 8. Why does Reno's `cwnd` in round 9 differ from Tahoe's?

B3. Tahoe is in slow start in round 9 but Reno is in congestion avoidance. Explain the difference in the rate at which the two windows grow in rounds 9–12.

B4. At what round does Tahoe's `cwnd` reach `ssthresh` after the round-8 loss? Why does Tahoe switch to congestion avoidance at that point?

B5. A timeout occurs in round 16 for both protocols. Compute the new `ssthresh` and `cwnd` for each protocol.

B6. Suppose instead that the round-16 loss were a triple duplicate ACK (not a timeout). Recompute `ssthresh` and `cwnd` for Tahoe and for Reno, and state the round-17 window for each.

B7. Compute the total number of segments sent by each protocol in rounds 1–16. Which protocol delivers more data by the end of round 16? By how much?

B8. Suppose the network had a *second* loss in round 10 (two segments of the same window lost). Discuss how Reno and Tahoe would each respond, and why this scenario is a weakness of basic Reno.

B9. Suppose the losses are random (for example on a wireless link) and not caused by congestion. Which protocol suffers more? Explain using the AIMD reaction to each loss type.

B10. Consider a long-lived flow whose window oscillates between $W/2$ and $W$ (ideal sawtooth). Show that the average window is $0.75\,W$ and hence that the average throughput is approximately $\dfrac{0.75\,W \cdot \mathrm{MSS}}{\mathrm{RTT}}$.

B11. Two Reno flows with different RTTs share one bottleneck. Which flow obtains more bandwidth, and why? Relate your answer to how `cwnd` grows per RTT.

B12. Draw (by hand or in TikZ) the `cwnd` trace for Tahoe and Reno if the triple duplicate ACK occurs in round 12 instead of round 8, keeping the timeout in round 16. Explain any change in the protocols' relative behaviour.

## Part C: Textbook Problems 32–44 (Chapter 3)

**P32.** Consider the TCP procedure for estimating RTT. Suppose that $\alpha = 0.1$. Let $\texttt{SampleRTT}_1$ be the most recent sample RTT, let $\texttt{SampleRTT}_2$ be the next most recent sample RTT, and so on.

(a) For a given TCP connection, suppose four acknowledgments have been returned with corresponding sample RTTs: $\texttt{SampleRTT}_4$, $\texttt{SampleRTT}_3$, $\texttt{SampleRTT}_2$, and $\texttt{SampleRTT}_1$. Express `EstimatedRTT` in terms of the four sample RTTs.

(b) Generalize your formula for $n$ sample RTTs.

(c) For the formula in part (b), let $n \to \infty$. Comment on why this averaging procedure is called an *exponential moving average*.

**P33.** In Section 3.5.3, we discussed TCP's estimation of RTT. Why do you think TCP avoids measuring the `SampleRTT` for retransmitted segments?

**P34.** What is the relationship between the variable `SendBase` in Section 3.5.4 and the variable `LastByteRcvd` in Section 3.5.5?

**P35.** What is the relationship between the variable `LastByteRcvd` in Section 3.5.5 and the variable $y$ in Section 3.5.4?

**P36.** In Section 3.5.4, we saw that TCP waits until it has received three duplicate ACKs before performing a fast retransmit. Why do you think the TCP designers chose not to perform a fast retransmit after the first duplicate ACK for a segment is received?

**P37.** Compare GBN, SR, and TCP (no delayed ACK). Assume that the timeout values for all three protocols are sufficiently long such that five consecutive data segments and their corresponding ACKs can be received (if not lost in the channel) by Host B and Host A respectively. Suppose Host A sends five data segments to Host B, and the second segment (sent from A) is lost. In the end, all five data segments have been correctly received by Host B.

(a) How many segments has Host A sent in total and how many ACKs has Host B sent in total? What are their sequence numbers? Answer this question for all three protocols.

(b) If the timeout values for all three protocols are much longer than 5 RTT, which protocol successfully delivers all five data segments in the shortest time interval?

**P38.** In our description of TCP in Figure 3.53, the threshold `ssthresh` is set as $\texttt{ssthresh} = \texttt{cwnd}/2$ in several places and is referred to as being set to half the window size when a loss event occurred. Must the rate at which the sender is sending when the loss event occurred be approximately equal to `cwnd` segments per RTT? Explain your answer. If your answer is no, can you suggest a different manner in which `ssthresh` should be set?

**P39.** Consider Figure 3.46(b). If $\lambda'_{in}$ increases beyond $R/2$, can $\lambda_{out}$ increase beyond $R/3$? Explain. Now consider Figure 3.46(c). If $\lambda'_{in}$ increases beyond $R/2$, can $\lambda_{out}$ increase beyond $R/4$ under the assumption that a packet will be forwarded twice on average from the router to the receiver? Explain.

**P40.** Consider Figure 3.61, which shows TCP window size as a function of time (transmission rounds 0–26, `cwnd` up to about 45 segments). Assuming TCP Reno is the protocol experiencing the behavior shown, answer the following. Justify each answer briefly.

(a) Identify the intervals of time when TCP slow start is operating.

(b) Identify the intervals of time when TCP congestion avoidance is operating.

(c) After the 16th transmission round, is segment loss detected by a triple duplicate ACK or by a timeout?

(d) After the 22nd transmission round, is segment loss detected by a triple duplicate ACK or by a timeout?

(e) What is the initial value of `ssthresh` at the first transmission round?

(f) What is the value of `ssthresh` at the 18th transmission round?

(g) What is the value of `ssthresh` at the 24th transmission round?

(h) During what transmission round is the 70th segment sent?

(i) Assuming a packet loss is detected after the 26th round by the receipt of a triple duplicate ACK, what will be the values of the congestion window size and of `ssthresh`?

(j) Suppose TCP Tahoe is used (instead of TCP Reno), and assume that triple duplicate ACKs are received at the 16th round. What are `ssthresh` and the congestion window size at the 19th round?

(k) Again suppose TCP Tahoe is used, and there is a timeout event at the 22nd round. How many packets have been sent out from the 17th round until the 22nd round, inclusive?

**P41.** Refer to Figure 3.55, which illustrates the convergence of TCP's AIMD algorithm. Suppose that instead of a multiplicative decrease, TCP decreased the window size by a constant amount. Would the resulting AIAD algorithm converge to an equal-share algorithm? Justify your answer using a diagram similar to Figure 3.55.

**P42.** In Section 3.5.4, we discussed the doubling of the timeout interval after a timeout event. This mechanism is a form of congestion control. Why does TCP need a window-based congestion-control mechanism (as studied in Section 3.7) in addition to this doubling-timeout-interval mechanism?

**P43.** Host A is sending an enormous file to Host B over a TCP connection. Over this connection there is never any packet loss and the timers never expire. Denote the transmission rate of the link connecting Host A to the Internet by $R$ bps. Suppose that the process in Host A is capable of sending data into its TCP socket at a rate $S$ bps, where $S = 10R$. Further suppose that the TCP receive buffer is large enough to hold the entire file, and the send buffer can hold only one percent of the file. What would prevent the process in Host A from continuously passing data to its TCP socket at rate $S$ bps? TCP flow control? TCP congestion control? Or something else? Elaborate.

**P44.** Consider sending a large file from one host to another over a TCP connection that has no loss.

(a) Suppose TCP uses AIMD for its congestion control without slow start. Assuming `cwnd` increases by 1 MSS every time a batch of ACKs is received and assuming approximately constant round-trip times, how long does it take for `cwnd` to increase from 6 MSS to 12 MSS (assuming no loss events)?

(b) What is the average throughput (in terms of MSS and RTT) for this connection up through time = 6 RTT?